Pioneering the technologies that are bridging the gap between printed information and the online world

PrXML
Preservation Markup Language

In Brief

Version 5
April 2005

PrXML - Preservation Markup Language in Brief
Document Version 5

April 2005

Olive Software Inc.
2170 South Parker Road
Suite 290
Denver, Colorado
80231

Tel: 1-720-747-1220
Fax: 1-720-747-1217
E-mail: info@olivesoftware.com

Table of Contents

1 Olive's Content Uniformity Concept

In today's world, there is an ever growing demand for quick and easy access to all types of information generally archived by a variety of methods like paper, microfiche, HTML, PDF, etc.

Some emerging content management and search engine technologies have been developed to bring order and structure to this information chaos. The challenge of these recent technologies is that information is still managed as a large, hard to utilize collection of different property formats and objects, some of which are incompatible and dependant upon proprietary applications, databases and software architectures.

In order to really utilize content and turn it into valuable, easy to access knowledge assets, we must have a technology framework that streamlines the content into a single, uniform, non-proprietary manageable format. It must be based on an open architecture that enables future-proof preservation and knowledge collaboration.

Olive has developed a breakthrough technology that provides automatic transformation of documents and content into a single, unified intelligent XML format. Olive's intelligent content model enables unified view and access interfaces, true-to print, component access or customized formatting, content portability and future-proof preservation.

2 PrXML – Preservation Markup Language

2.1 The Long-Term Content Preservation Problem

The traditional problem that has plagued content preservation technologies is the dependency of content on the technologies from which they were created and more specifically the "proprietary binary document formats". The organization's ability to access and read archived digital content in the future mainly depends on the software vendor's willingness and ability to constantly maintain backward compatibility of such documents associated reading applications.

Many software developers frequently introduce new proprietary document formats, (such as MS Word and PDF), mainly to increase the customer-vendor technology dependency. Problems arise from a variety of reasons such as when support is discontinued with versions of proprietary applications, or when organizations decide to stop using certain software. This usually results in lack of compatibility between older, archived digital documents diminishing access capabilities to the content with current software. For example, it is virtually impossible for today's word processors to open files created only 15 years ago.

Another aspect of the long-term digital preservation challenge is not only the binary files, but also the digital archive applications themselves. Most of the existing document management and archiving systems are based on a complex, multi-applications, distributed model. This model is based upon management of a variety of proprietary file formats stored in a proprietary repository, while at the same time, the document's metadata is stored in a proprietary database or management application. It is virtually impossible to maintain such a complex, unsynchronized model over many years.

2.2 Olive XML Enables Future-proof Preservation

Olive's digital preservation model is the only model that provides a solution for the critical challenge of long-term digital document preservation. The solution is based on the idea of a complete separation between the content and its corresponding technology and data base free collections and metadata management. Rather than dependency on risky binary formats and on proprietary binary data base that maintains the metadata index separately, Olive converts and encapsulates the documents and their metadata into unified XML repository, stored in a platform independent file system.

XML is the only format that can guarantee long-term document preservation, since it is an ASCII-based, open-standard, free of proprietary binary codes and supported by all the major software vendors.

Olive's XML content preservation model increases the longevity of content protected from technological changes. This ensures that the content will be readable by any web browser, portable and compatible with any operating system and hardware platform both today and in the future. Olive's XML was adopted and is currently being used by the On-line Computer Library Center (OCLC) and hundreds of publishers, enterprises and libraries around the world.

2.3 The PrXML Concept

Olive's XML architecture is based on our Preservation Markup Language schema (PrXML). The PrXML schema is principally a comprehensive page and document description language that maps the original document's content, style and hidden intelligence in an open source XML format.

PrXML encapsulates and preserves text, metadata, structure, context, relationships, styling data, file properties, knowledge tags, hyperlinks, graphics and image maps. The highly rich tagging nature of the PrXML schema enables the ability to restore the look and feel of the original documents (like PDF does) and at the same time provides flexibility to create customized, flexible views on demand and advanced search capabilities (the XML main value), all within any standard browser.

PrXML has been designed as a physical "Hyper Schema". A "Hyper Schema" is a raw XML schema that is not limited to a specific industry or domain but is situated "above" other schemas. By using a "Hyper Schema", users can easily transform content from the raw PrXML into any other logical industry schema standards on demand. This is easily accomplished by utilizing common filters and XML transformations (XSL). Known XML schemas for example, are NITF, OAI, NewsML, ADSML, METS, PRISM-PAM and more.
The advantage of this model is the ability to create an XML infrastructure that is not limited to a specific industry schema today, but open to be adjusted on the fly to specific schemas as needed now or in the future. Another distinct advantage is the ability to have one system supporting multiple, specific schemas in parallel.

2.4 The Contextual Information Component

The Contextual Information Component (“Contextual Component “or “Entity”) is another unique aspect of PrXML intelligence. Almost every document is a collection of different linked ideas. Contextual Components represents these ideas and are the logical units (or atoms) of information within a given document that define these ideas. The rules that expose the Contextual Components in a given document are defined by the information owner or by the specific content application. For example, a component can be an article in a journal or newspaper, a picture in a magazine or a section or a paragraph in a report.

Most users are regularly seeking specific ideas in documents and dislike searching and downloading large documents, just for reading a certain item in it. Contextual Component extraction and tagging enables management and direct search and access to any information nugget within a document, independent of other components instead of downloading large files. At the same time, the content components are always maintained in the context of its original document.

Information Components also improves the search engine indexing process itself, thus increasing the relevance ranking of search results. For example, if a user searches all articles that are related to the key words "food", "oil" and "olive". Regularly he will get all of the PDF documents that include such key words even if they are not related to each other, (for example a page that consists of one article about "Olive trees and gardening" and another article that talks about "Oil fields in Iraq"). With Information Component based indexing and search,, the retrieved search results will include only articles or advertisements (component examples) that include those specific key words in the component space.

2.5 Image Mapping and Indexing

Effective access and search of historical documents generated from paper and microfilm is an important part of the PrXML effort. Scanned images of such materials are usually degraded and are difficult to process and convert to text with regular Optical Character Recognition (OCR) technologies, resulting corrupted text files.

Principally, the readability accuracy factor could greatly increase by allowing users to access and read the images directly instead of reading poorly OCR generated text. The task of comprehending the degraded text is then transferred to the human eye and brain—which is the best OCR.

However, what about the search factor in this case? How can we search for such images?
Olive unlinking of the 'searchability' factor from the 'readability' by a patented technique called Bitmap Indexing™. This technique is based on the mapping of each meaningful group of pixels (containing a page element like a title, body text, word-patterns or picture) on the page image into the PrXML. Indexing such valuable image mapping data, enables direct search and access to and sophisticated manipulation of image "clips" instead of cumbersome access to the page image as a whole.
Bitmap Indexing results in meaningful end user features, for example, search hits can be highlighted in an article image and search result pages can display scaled images of components, not imperfect OCR text.

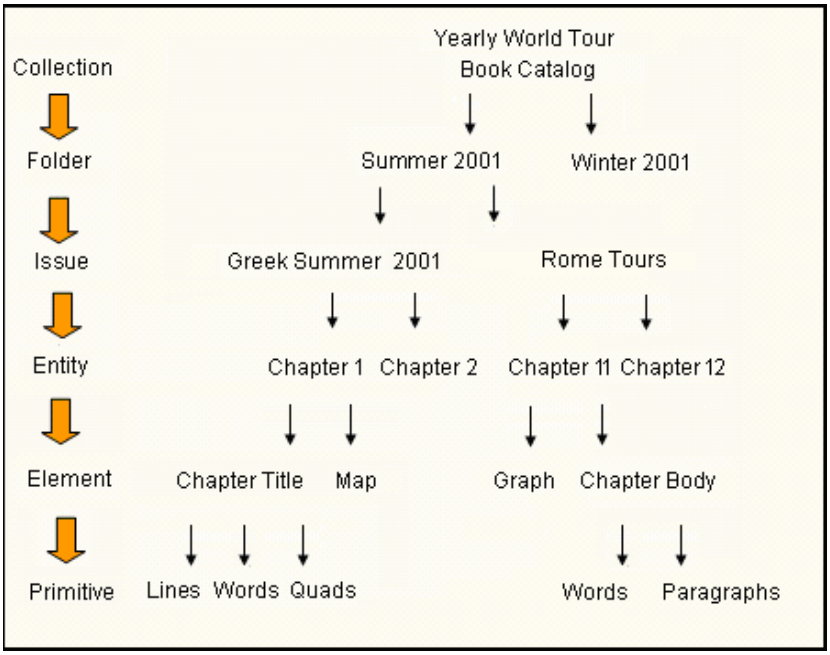
Olive's PrXML also defines special word pattern tags called Adaptive Probability Fuzzy Search (APFS) (Patent Pending) tags. These word patterns define the quality parameters and the probability for suspected OCR errors in each word. Smart indexing of these tags enables a search engine to effectively search within degraded images, while compensating on possible OCR mistakes. Without APFS, the regular fuzzy search feature that normally exists in many search engines, will provide a mass of irrelevant search results because the fuzzy search is performed on the entire document text instead of just on the suspect misspelled words.

2.6 The PrXML Structure

Contextual Components (Entities) contain their own internal structure - "Elements". For example, in the case of a newspaper, if a component is defined as an "article" then its elements might be a headline, roof, drop head and byline. In the case of a book, an entity might be a chapter so its elements would be a chapter title, body and graphs. An element is still a unit of information but it takes a collection of elements together, to define a complete Entity.

A further breakdown of the structure includes the elements "nuclear elements" called "Primitives". Primitives are the fundamental building blocks of the PrXML schema. A Primitive is basically a physical definition (not logical definitions like Elements and Components) that defines a rectangular region on a page that contains text or graphic information such as a word or paragraph.

The metadata tags in each Component's XML, collects knowledge about the Entity's environment and its relationships to other components; information like the section the Component belongs to, the page it is on, its size, the articles it is related to, etc. Each "word" XML tag contains attributes such as page coordinates and font style information (supplied as *Font Style Gallery* references).



The global Font Style Gallery contains descriptions of all the font styles (comprised of font types, font styles, sizes and size ranges) in the original document and enables the restoration of its original look and feel as needed. The Style Gallery is common to all Entities and is represented as an external unit in the XML of each print issue remaining unique to the edition, catering to the dynamic nature of print design.

Fast navigation to specific information components, requires detailed hierarchical structures that give direct access to different PRXML repository abstraction levels.

PRXML provides 3 coherent hierarchy mechanisms:

- External hierarchy is provided as a "folder taxonomy tree" whereby each document or entity can be applied to one, or a few nodes within this tree. Multiple taxonomies may also be provided.
- Document structure hierarchy is provided by the document Table of content (TOC.xml), describing the relations between components (entities) within a document.
  For example, Scientific Magazine contains different articles (entities), which are usually organized within a hierarchical structure.
- Each Information Component (IC) has a structure of its own, which defines different elements and their reading order. IC's may be embedded one within the other. Usually, an embedded IC is an object that disturbs the reading order (such as, tables, pictures, leads, etc.).

2.7 Custom Metadata

PRXML defines metadata on two different levels – at a document level and contextual component (entity) level.

Both metadata sets are open for defining new, additional fields that may be defined per specific application. Dublin core metadata standard is used as the default metadata.

2.8 Custom In-line Tags

PrXML comes with a pre-defined set of in-line tags and allows you to define new inline tags. This family of tags, is also known as Primitive Tags, because they markup part of a primitive. Primitive tags can be defined per application. For example, Photographer, Agency, Company, Address, Location, Core Phrase (summary) etc. Olive XML Distiller software provides powerful functionality to recognize primitive tags automatically, based on rules and statistics. Third party engines may be used to recognize and mark primitive tags on already published PrXML.

3 The Olive XML Repository

3.1 The XML Repository Concept

The Olive XML Repository™ is a simple file system based repository (windows or UNIX file systems), designed for data-base free, platform independent, long-term content preservation and archiving. The XML files that are stored in the Olive XML Repository are complemented by images that are snapshots of the original printed representation of each information object. The XML Repository contains a set of images representing the entire print issue, piece by piece, in different resolutions and formats. These images are a powerful tool when combined with XML components, elements and primitives, which detail each image's role and position in the original printed page. Olive XML Repository can also store rich media objects such as video clips, animated gifs and sound.

3.2 The XML Repository General Structure

The Olive Repository is a hierarchical file system, structured as a tree of files and folders that reside in the Repository root directory. The following is an example that describes a newspaper archive repository:

The Root directory contains a separate folder for each publication and two special folders for internal use. These are common Files and common Styles (see Fig. 1).

A Publication directory bears a unique name given to that publication (e.g. “Olive Weekly”).

In the case of periodicals, issues are stored in the date that they are published.

- Each Year directory (possessing a four-digit name), contains Month subdirectories
- Each Month directory (possessing a two-digit name 01 to 12), contains day subdirectories
- Each Day directory (possessing a two-digit name 01 to 31).
  In archives the following information is contained in a zip file:
  - An XML Table of Contents file named – TOC.xml
  - A Font Style Gallery file named – StyleGallery.txt
  - A copy of the source document (PDF, TIFF, etc…) file carrying the name of the original file
  - The page directory contains the following:
  - IMG directories
  - Primitive images
  - Scaled page images (in one or a few resolutions)
- Page thumbnails
- XML Entity files
- XML Page file

The name of the page directory is the same as the physical page number it represents (without trailing zeroes).

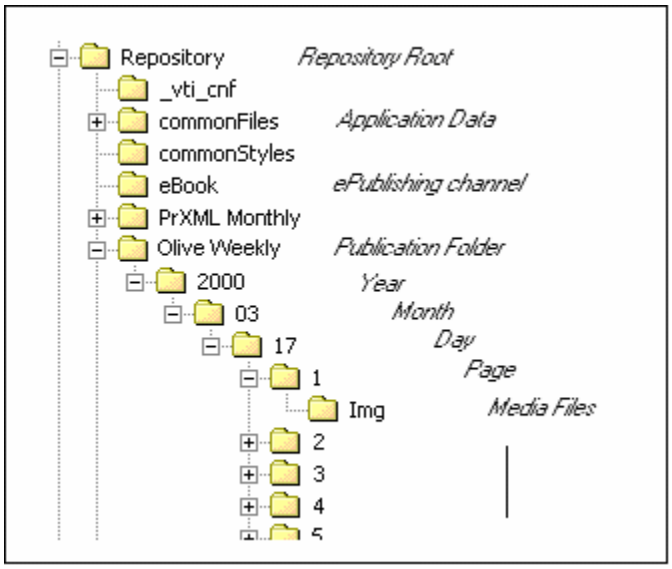


Figure 1: PrXML Periodical Repository – Sample directory tree

In a case of non-periodicals, documents are stored according to an Olive unique code in a three layered hierarchal fashion (similar to that of the periodical).

The third level in the hierarchy contains the following information (for archives only) packaged in a zip file:

- A Table of Contents XML file named TOC.xml
- A Font Style Gallery file named StyleGallery.txt
- A copy of the source PDF file, given the name of the original file
- Per page and IMG directories are identical to the structure in the previously mentioned periodical repository.
  Users can find files describing the entities, elements and primitives in a page folder.

3.3 Standards

3.3.1 File Formats

The PrXML image repository is a coherent set of files of different formats. These currently consist of XML and XSL files as well as of corresponding media files such as PDF, GIF, JPG, PNG etc. that are used in conjunction with the XML files (see also 3.4, “Auxiliary Data” below).
All files are generated according to the respective format standards.

3.3.2 XSL Language

XSL scripts allow for automatic transformation of Olive XML content into formats like HTML, ASCII, WAP and more. These files vary according to application demands, and may allow either batch conversion of XML or on-the-fly content display in various formats and customized views.
The XSL language used in conversion of PrXML and XML is XSLT. XSL is currently supported by MS Internet Explorer, as well as by a majority of XML processors. XSLT is becoming a de-facto standard for the XML industry.

3.4 Auxiliary Files and Data

Images

Images play an important role in the PrXML Repository. Page image snapshots for example, are created and used not only to preserve and present historical scanned documents but also to preserve and present the original view of new documents, minimizing the dependency on special font availability at the client side.

Olive can store every image in a document, in multiple resolutions and formats. By default all page snapshot images are stored as 256-color GIF or PNG image format of various resolutions, while color photos are stored in JPEG format with various compression levels.

The decision on how images are stored is based on a smart algorithm that calculates the best method of generating images based on the image content. This algorithm takes into account the storage space the image takes up as well as the image’s clarity requirements. For example, a character or symbol is best stored as a gif or png. If a jpeg was used instead, the character would not be clear, while pictures that contain many colors are better stored in jpeg format to insure the pictures quality.

Furthermore, in some instances such as with newspapers, images are stored in a few different resolutions. This allows for readers to choose the image size most fitting to their screen resolution and or network speed. The Repository contains three kinds of images:

- Page thumbnails
- Page snapshot images
- Primitive images

Newspaper (A2/A3) Example of Image Type Characteristics

Image Type	Naming convention	Typical resolution	Typical size (From Digital PDF)	Typical size (From post-OCR TIFF)
Page Thumbnail	pv<PageNum>.gif (ie. pv001.gif)	15dpi	5-7kb	5-15kb
Average Page Image (of five resolutions)	pg<ImageNum>.gif (ie. pg124.gif)	30-45dpi	20-30kb	80-100kb
Elements/Primitives	pc<PictureNum>.gif (ie. pc00140.gif)**	60-120dpi	5-8kb	20-30kb

* The typical newspaper page consists of 20-30 elements. In the case of newspapers predating 1950, this number could rise to as much as 70-100.

** A picture is just one example of an element. Other elements include: an article (ar<articlenum>.gif) and an advertisement (ad<AdNumber>.gif).

Document (A4) Example of Image Type Characteristics

Image Type	Naming convention	Typical resolution	Typical size (From Digital PDF)	Typical size (From post-OCR TIFF)
Page Thumbnail	pv<PageNum>.gif (ie. pv001.gif)	15dpi	2-5kb	5-15kb
Page Image	pg<ImageNum>.gif (ie. pg124.gif)	30-45dpi	20-40kb	20-120kb
Elements/Primitives	ar<ArticleNum>.gif (ie. ar00140.gif)**	60-120dpi	0.5-2.5kb	20-30kb

** A picture is just one example of an element. Other elements include: an article (ar<articlenum>.gif) and an advertisement (ad<AdNumber>.gif).

Image Type Usage

The set of images present in the repository allows for the building of a document in its original look and feel.

- ***Page thumbnails*** can represent the general look of an entire issue or section, or can be used to graphically represent search results
- ***Page snapshot images*** are used to simulate the original document look and feel in the browser reading application
- ***Primitive images*** are used to simulate the original view of a page component such as article or article elements.

PDF Documents

The original documents format such as MS Word, PostScript or TIFF, are converted and stored in the Olive repository as unified PDF files. This is in order to create a set of homogenic files which may be needed for download, print and PDF delivery purposes.

3.4.1 XML and XSL

Aside from the entity XML files that represent the content itself, the repository contains XML files that serve as auxiliary data. These describe the Repository's structure and provide additional sources of information used by the client applications:

- Documents XML (Table Of Contents)
- Section XML
- Authors dictionary
- Predefined collections description
- Keywords

3.5 Storage Size Considerations

The Repository size may vary, and depends on a number of factors. These include page size, number of pages, and usage of graphics, color and page layouts. The numbers below are averages provided as a rough guideline to the necessary storage calculation. It is important to note that repository size is flexible and can be optimized by the ability to adjust the image resolution levels and compression methods.

3.6 Born Digital Source Materials

Below are the page size approximations, of example documents processed by Olive Software:

Format	A3 (Newspaper Size)	A4 (Standard Size)
PDF	100-200K	20-50K
XML	30-70K	8-16K
Images	150-250K	40-65K
Total (Approximate)	400K	100K

A newspaper archive for example, consisting of a year's issues (300 issues multiplied by 50 pages) would require approximately 6 GB of storage space.

While an archive of documents at about the same size would require approximately 1.5 GB of storage space.

3.7 Scanned Source Materials

A typical newspaper page (A3) derived from TIFF's has a total size of 2-4 MB; while a typical document page (A4) derived from TIFF's has a total size of 1 MB.

A year's newspaper archive (300 issues multiplied by 20 pages) would therefore require approximately 12-24 GB plus 10-20 % for digital source materials (see above). The comparable standard document archive of the same size will be about 3.5-7.5 GB plus 10-20 % for digital source materials.

NOTE: Due to its hierarchical structure, the Repository can easily be divided into volumes and distributed among different hard-drives or storage devices.

4 The Entity XML Structure (Newspaper Example)

4.1 Entity Types

- Article
- Ad
- Picture (this refers to a standalone picture not associated with an article).

4.2 Element Hierarchy

<XMD-Entity>

<Meta>

<Link>

<Application Data>

<HedLine_hl0>
<Primitive>
<L>
<W>
<Q>
<q>
<P>
<BOX>
<SubElement>
<L>
<W>
<Q>
<q>
<P>

<HedLine_hl1>
<Primitive>

<HedLine_hl2>
<Primitive>

<Byline>
<Primitive>

<Content>
<Primitive>

<Lead>
<Primitive>

<Continuation_To>
<Primitive>

<Continuation_From>
<Primitive>

<Continuation_Content>
<XMD-Entity>

<Picture>
<Caption>
<Primitive>
<Byline>
<Img>
<Primitive>

<Table>
<Caption>
<Primitive>

<Supplementary>
<Primitive>

<Decoration>
<Img>
<Hor>
<Ver>
<Frame>
<Gallery_Img>

<StyleGallery>
<Style>

4.3 Alphabetical Element List

Tag	Description
<Application_Data>	Extensible application communication element
<Byline>	Authors element
<Caption>	Picture or table caption
<Content>	Body text element
<Continuation_Content>	Element containing this entity's continuation text (or the text it continues from)
<Continuation_From>	"Continuation from page" reference element
<Continuation_To>	"Continuation to page" reference element
<Decoration>	Element used for decorative graphic elements in the entity, like dividers and frames
<Frame>	Frame decoration element
<Gallery_Img>	Element reserved for future
<HedLine_hl0>	Roof-title element
<HedLine_hl1>	Main title element
<HedLine_hl2>	Subtitle element
<Hor>	Horizontal line decoration element
<Img>	Image element
<L>	Text line element (appears before a new line)
<Lead>	Lead element
<Meta>	Metadata element
<P>	Paragraph element (will appear before the start of a new paragraph)
<Picture>	Element used for standalone pictures (not associated with articles)
<Primitive>	Element representing a rectangular region of the page together with its contents
<Q>	Quad element – used for part of a word in cases of words broken by hyphens, line-breaks, etc. (this element is used for the beginning of a word)
<q>	Quad element – used for part of a word in cases of words broken by hyphens, line-breaks, etc. (this element is used for the

Tag	Description
	end of a word)
<Style>	Font style element
<StyleGallery>	Font style gallery element
<SubElement>	Element used for a unique part of a primitive
<Supplementary>	Redundant element for unrecognized entity parts
<Table>	Table element
<Ver>	Vertical line decoration element
<W>	Word element
<XMD-entity>	Main tag
APFS tags	

5 Page XML Structure

5.1 Element Hierarchy

<XMD-Page>

<Meta>

<Link>

<Entity>

<Primitive>

5.2 Alphabetical Element List

Tag	Description
<Entity>	Element containing a single Entity XML
<Link>	Element containing links to next and previous sections
<Meta>	Element containing metadata pertaining to the page
<Primitive>	Element description
<XMD-Page>	Main tag

6 Table of Contents XML Structure

6.1 Element Hierarchy

<XMD_TOC>

<Head_np>

<Meta>

<Link>

<SOURCE>

<Body_np>

<Section>

<Page>

<Entity>

6.2 Alphabetical Element List

Tag	Description
<Body_np>	Element containing all the sections of the newspaper issue
<Entity>	Element containing one entity XML
<Head_np>	Element containing metadata and a link to the source PDF
<Link>	Element containing link to the source PDF
<Meta>	Element containing metadata related to the newspaper issue
<Page>	Element containing all the entities in one newspaper page, which should be sorted by size and type, respectively
<Section>	Element containing one newspaper section